

PROJET : DAILY PLANZ

07/04/2026

Équipe :2

OUISSAL NOURI

HOUDA EL MKKADEM

NADA SARHANE

Chef de projet

Product Owner

Développeur

Contenu du document :

Livrable 1 – Mini-Cahier des Charges

Livrable 2 – Work Breakdown Structure (WBS)

Livrable 3 – Estimation du projet

Livrable 4 – Diagramme de Gantt

Enseignant :

Pr. Sanae EL MIMOUNI

Méthodologies de Développement Logiciel – 2025-2026

1. Livrable 1 Mini-Cahier des Charges (Mini-CDC)

1. Contexte

Aujourd'hui, avec le développement de la technologie, beaucoup de personnes n'utilisent plus les cahiers pour organiser leurs tâches. Elles préfèrent utiliser un **site web** simple et pratique pour noter ce qu'elles doivent faire et suivre les tâches terminées.

Dans ce cadre, notre projet **Daily planz** est un **site web** de productivité qui vise à résoudre les problèmes de mauvaise organisation, d'oubli des tâches et de manque de suivi des tâches accomplies. Il permettra à l'utilisateur de gérer facilement ses tâches personnelles dans un

seul
espace.

Cette solution peut faire gagner du temps à l'utilisateur, avec un accès à ses tâches en moins de 10 **secondes**, et une réduction estimée jusqu'à **30 %** du temps perdu dans le rappel ou la recherche des tâches.

2. Parties prenantes

Partie prenante	Rôle	Besoins	Implication
Université	Client fictif	Suivre un projet clair et réalisable Organiser ses	Moyenne
Utilisateur final	Utilisateur principal du site	tâches facilement	Élevée
Administrateur système	Gérer le site et les données	Disposer des droits de gestion et de maintenance	Moyenne
Chef de projet	Coordonner le travail de l'équipe	Assurer le respect du planning et des livrables	Élevée
Équipe de développement	Concevoir, développer et tester le site	Avoir des besoins clairs et des outils adaptés	Élevée

3. Périmètre fonctionnel (MoSCoW)

Priorité	Fonctionnalité
Must	Créer un compte
Must	Se connecter

Must	Ajouter une tâche
Should	Modifier une tâche
Should	Supprimer une tâche
Could	Marquer une tâche comme terminée
Could	Rechercher une tâche
Won't	Partager des tâches avec d'autres utilisateurs

4. Contraintes

Le projet doit respecter plusieurs contraintes. Sur le plan technique, le site web sera développé avec **HTML, CSS, JavaScript, Java Spring Boot** et **MySQL**. Sur le plan du délai, le projet doit être réalisé selon le planning prévu jusqu'à la séance **S12**. Enfin, sur le plan de la sécurité, il faut assurer la protection des comptes utilisateurs, la validation des données saisies et l'application de vérifications de sécurité comme **SAST** et **DAST**.

5. Dépendances externes

Le projet dépend de certains outils et services externes. Il repose principalement sur **MySQL** pour le stockage des données, sur **GitHub** pour l'hébergement du code source et sur un **navigateur web** pour l'accès au site. En cas d'indisponibilité de la base de données, les utilisateurs ne pourront pas accéder à leurs comptes ni à leurs tâches. De même, une indisponibilité de GitHub peut ralentir le travail de l'équipe.

6. Cas d'utilisation

Cas 1 : Créer un compte

- **Acteur** : Utilisateur
- **Précondition** : L'utilisateur ne possède pas encore de compte
- **Déroulement** : Il saisit son email et son mot de passe, puis valide l'inscription
- **Résultat** : Le compte est créé avec succès
- **Cas d'erreur** : Email déjà utilisé ou mot de passe invalide

Cas 2 : Se connecter

- **Acteur** : Utilisateur
- **Précondition** : L'utilisateur possède déjà un compte
- **Déroulement** : Il saisit son email et son mot de passe, puis clique sur connexion
- **Résultat** : Il accède à son espace personnel
- **Cas d'erreur** : Identifiants incorrects

Cas 3 : Ajouter une tâche

- **Acteur** : Utilisateur connecté
- **Précondition** : L'utilisateur est connecté au site
- **Déroulement** : Il saisit une nouvelle tâche puis l'enregistre
- **Résultat** : La tâche est ajoutée à sa liste
- **Cas d'erreur** : Champ vide ou erreur d'enregistrement

7. Architecture technique

L'architecture technique du projet repose sur trois couches principales. La première couche est le **front-end**, développé avec **HTML, CSS et JavaScript**, qui permet l'interaction avec l'utilisateur. La deuxième couche est le **back-end**, développé avec **Java Spring Boot**, qui gère la logique métier et les traitements. La troisième couche est la **base de données MySQL**, qui assure le stockage des comptes et des tâches. Cette architecture en trois couches permet une bonne organisation du site et facilite sa maintenance.

8. Risques

Risque	Probabilité	Impact	Stratégie de mitigation
Manque de temps	Moyenne	Élevé	Bien organiser le travail et respecter le planning
Difficulté technique	Moyenne	Élevé	Rechercher des solutions et demander de l'aide si nécessaire
Mauvaise communication	Faible	Moyen	Faire des réunions régulières
Problèmes avec GitHub	Moyenne	Moyen	Sauvegarder le travail régulièrement
Erreurs dans la base de données	Moyenne	Moyen	Tester souvent et vérifier les données

9. Glossaire

- **Tâche** : action que l'utilisateur doit réaliser.
- **Utilisateur** : personne qui utilise le site web.
- **Compte** : espace personnel créé par l'utilisateur.
- **Connexion** : action d'accéder au compte avec un email et un mot de passe.
- **Inscription** : création d'un nouveau compte.
- **Base de données** : système qui stocke les informations du site.
- **Administrateur** : personne qui gère le site et les données.
- **Statut** : état d'une tâche, par exemple terminée ou non terminée.

10. Hypothèses et exclusions

Nous supposons que l'utilisateur dispose d'un accès à Internet et d'un navigateur web pour utiliser le site. Nous supposons aussi que chaque utilisateur gère uniquement ses propres tâches personnelles.

En revanche, le projet ne prévoit pas de version mobile, de partage de tâches entre utilisateurs, ni d'intégration avec d'autres plateformes externes.

11. KPIs

- 100 % des fonctionnalités **Must** sont opérationnelles.
- L'utilisateur peut accéder à ses tâches en moins de **10 secondes**.
- L'ajout d'une tâche prend moins de **30 secondes**.
- Le site enregistre correctement les données dans la base **MySQL**.
- Aucun bug critique ne bloque l'inscription, la connexion ou l'ajout d'une tâche.

12. Planification macro

Phase	Période	Activités	Livrables
Cadrage	S1 à S2	Définition du projet et des besoins	Fiche projet, Mini-CDC
Conception	S3 à S4	Analyse, cas d'utilisation, architecture	Diagrammes, description technique
Organisation	S5 à S6	Préparation du backlog et du sprint 0	Product Backlog, Sprint Backlog
Développement	S7 à S10	Codage, tests, dépôt Git, CI	Version fonctionnelle du site
Finalisation	S11 à S12	Corrections, validation et préparation finale	Version finale et soutenance

13. Critères de succès

- L'utilisateur peut créer un compte sans erreur bloquante.
- L'utilisateur peut se connecter avec ses identifiants.
- L'utilisateur peut ajouter, modifier et supprimer une tâche.
- Les données sont correctement enregistrées dans la base de données.
- Toutes les fonctionnalités **Must** sont démontrées avec succès lors de la soutenance.

14. Wireframes

Les wireframes prévus couvrent les fonctionnalités prioritaires du site. Le premier écran représente la page **d'inscription**, avec les champs email, mot de passe et le bouton de création du compte. Le deuxième écran correspond à la page **de connexion**, avec les champs de saisie et le bouton de connexion. Le troisième écran représente la page principale **de gestion des tâches**, avec la liste des tâches, un champ d'ajout et les actions pour modifier ou supprimer une tâche.

LIVRABLE 2 : Work Breakdown Structure (WBS)

1. Arbre WBS

0. Projet : DailyPlanz

1. Spécification

1.1 Analyse des besoins

- 1.1.1 Identifier les fonctionnalités

- 1.1.2 Rédiger le cahier des charges

1.2 Conception

- 1.2.1 Concevoir l'architecture

- 1.2.2 Réaliser les wireframes

2. Développement

2.1 Backend

- 2.1.1 Développer API

- 2.1.2 Gérer base de données

2.2 Frontend

- 2.2.1 Créer interface utilisateur

- 2.2.2 Implémenter fonctionnalités

2.3 Authentification

- 2.3.1 Inscription

- 2.3.2 Connexion

3. Tests

3.1 Tests unitaires

- 3.1.1 Tester backend

- 3.1.2 Tester frontend

3.2 Tests fonctionnels

- 3.2.1 Vérifier fonctionnalités

- 3.2.2 Corriger bugs

4. Déploiement

4.1 Préparation

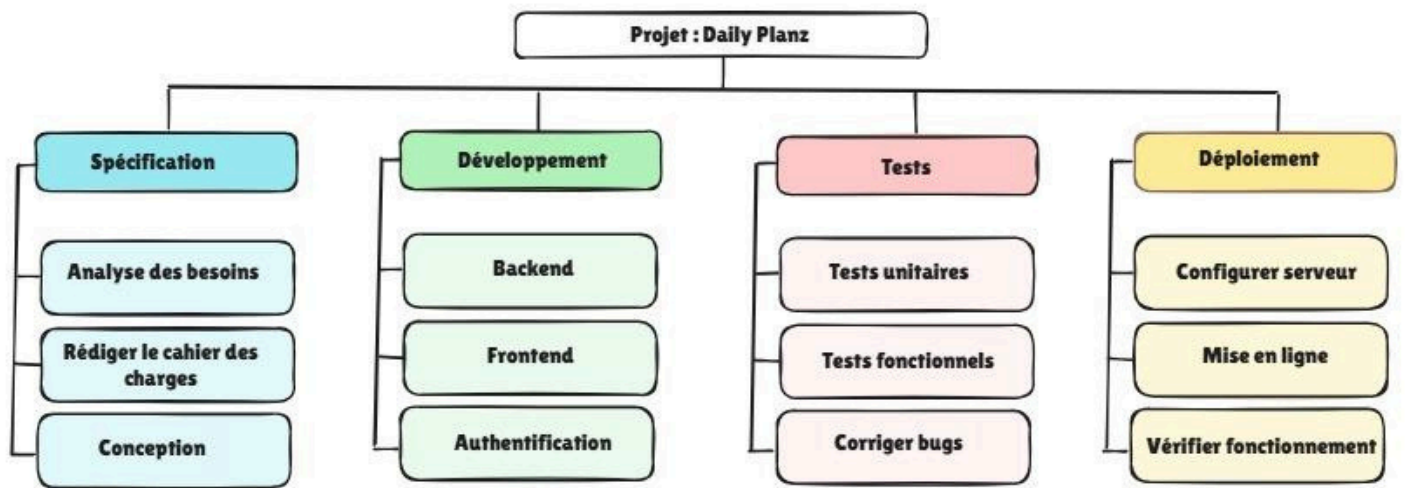
- 4.1.1 Configurer serveur

- 4.1.2 Préparer base de données

4.2 Mise en ligne

- 4.2.1 Déployer application

- 4.2.2 Vérifier fonctionnement



2. Dictionnaire WBS

Code	Livrable	Responsable	Durée estimée	Prédécesseur	Critère de fin
1.1.1	Liste des fonctionnalités du site	Équipe projet	2 jours	Aucun	Les fonctionnalités principales sont identifiées et validées
1.1.2	Cahier des charges rédigé	Chef de projet	2 jours	1.1.1	Le cahier des charges est complété et relu
1.2.1	Architecture technique du site	Développeur	1 jour	1.1.2	L'architecture du site est définie
1.2.2	Wireframes du site	Product Owner	1 jour	1.2.1	Les wireframes des écrans principaux sont réalisés
2.1.1	API développée	Développeur	3 jours	1.2.1	Les routes principales fonctionnent correctement
2.1.2	Base de données configurée	Développeur	2 jours	2.1.1	La base de données est créée et connectée au backend

LIVRABLE 3 : Estimation du projet

1. Méthode A : Estimation par analogie

Projet de référence : Application To-Do List simple

Durée du projet de référence : **2 mois**

Coefficients d'ajustement :

- Interface simple → 0.9
- Fonctionnalités similaires → 1
- Niveau débutant → 1.2

Calcul :

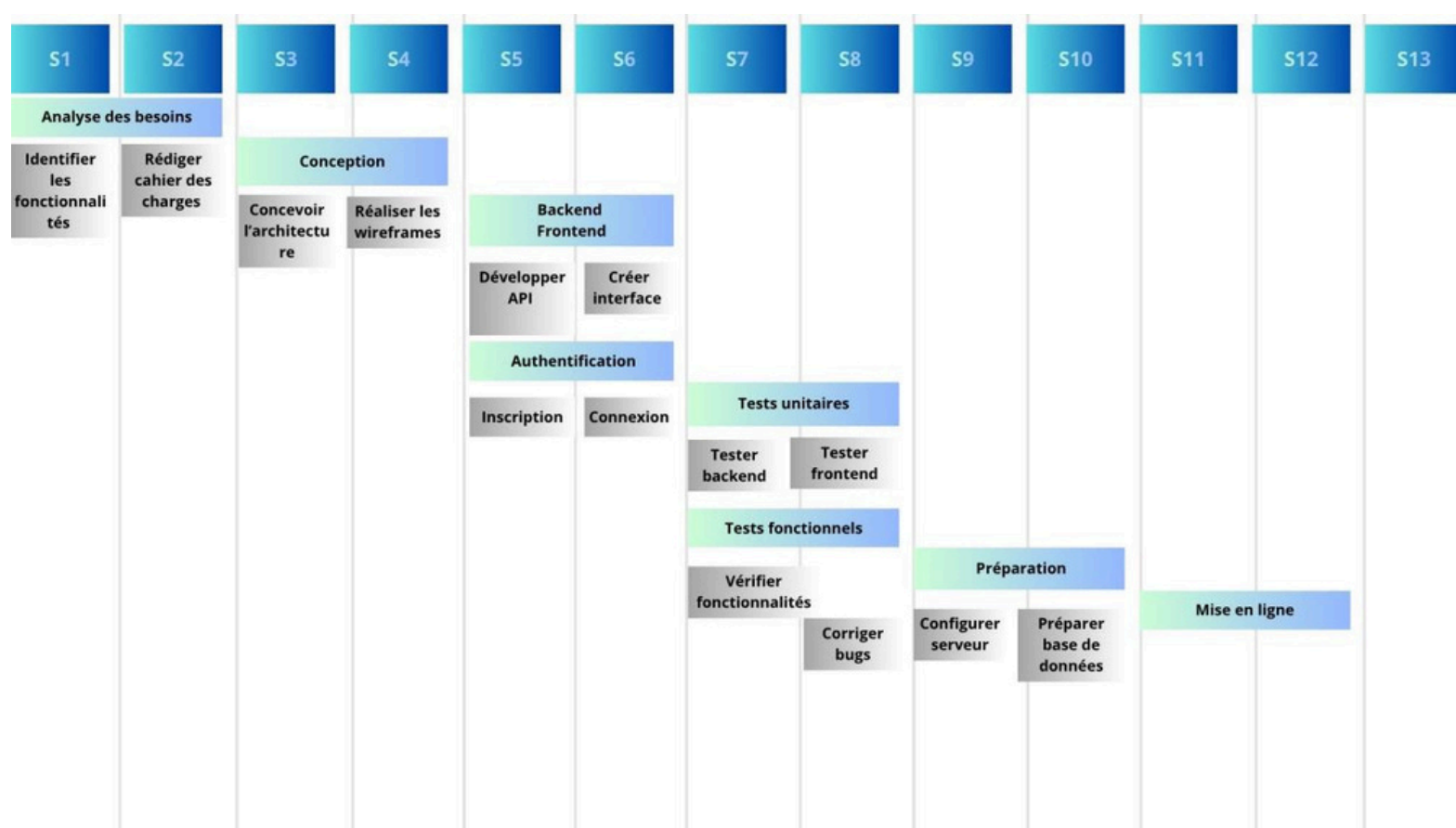
$$D_{\text{projet}} = 2 \times 0.9 \times 1 \times 1.2$$

$$D_{\text{projet}} = \mathbf{2.16 \text{ mois} \approx 2 \text{ mois}}$$

2. Conclusion

Ladurée retenue est **2mois** (méthode par analogie), car elle correspond mieux à la taille du projet et au calendrier académique.

LIVRABLE 4 : Diagramme de Gantt



1. Liste des tâches

Code	Tâche	Durée	Prédécesseur
1.1.1	Identifier les fonctionnalités	1 semaine	Aucun
1.1.2	Rédiger le cahier des charges	1 semaine	1.1.1
1.2.1	Concevoir l'architecture	1 semaine	1.1.2
1.2.2	Réaliser les wireframes	1 semaine	1.2.1
2.1.1	Développer l'API	2 semaines	1.2.1
2.2.1	Créer l'interface utilisateur	2 semaines	1.2.2
2.3.1	Développer l'inscription	1 semaine	2.1.1
2.3.2	Développer la connexion	1 semaine	2.3.1
3.1.1	Tester le backend	1 semaine	2.3.2
3.1.2	Tester le frontend	1 semaine	2.2.1
3.2.1	Vérifier les fonctionnalités	1 semaine	3.1.1, 3.1.2
3.2.2	Corriger les bugs	1 semaine	3.2.1
4.1.1	Configurer le serveur	1 semaine	3.2.2
4.1.2	Préparer la base de données	1 semaine	3.2.2
4.2.1	Mettre en ligne le site	2 semaines	4.1.1, 4.1.2

2. Parallélisme

Certaines tâches sont réalisées en parallèle dans le planning. C'est le cas de :

- **Développer l'API et Créer l'interface utilisateur ;**
- **Tester le backend et Tester le frontend ;**
- **Configurer le serveur et Préparer la base de données.**

3. Chemin critique

Le chemin critique du projet est la suite des tâches qui détermine la durée totale du planning. Dans ce projet, il est composé des tâches suivantes :

Identifier les fonctionnalités → Rédiger le cahier des charges → Concevoir l'architecture → Réaliser les wireframes → Créer l'interface utilisateur → Tester le frontend → Vérifier les fonctionnalités → Corriger les bugs → Configurer le serveur → Préparer la base de données → Mise en ligne

Les tâches du chemin critique ont une **marge totale égale à 0**.

4. Jalons

Les jalons retenus dans le projet sont :

- **Fin de l'analyse des besoins**
- **Fin de la conception**
- **Fin des tests**
- **Mise en ligne du site**

5. Tableaudes marges

Tâche	Durée	Début au plus tôt	Début au plus tard	Marge totale	Critique ?
Identifier les fonctionnalités	1 semaine	S1	S1	0	Oui
Rédiger le cahier des charges	1 semaine	S2	S2	0	Oui
Concevoir l'architecture	1 semaine	S3	S3	0	Oui
Réaliser les wireframes	1 semaine	S4	S4	0	Oui
Développer l'API	2 semaines	S5	S5	0	Oui
Créer l'interface utilisateur	2 semaines	S5	S5	0	Oui
Développer l'inscription	1 semaine	S6	S6	0	Oui
Développer la connexion	1 semaine	S6	S6	0	Oui
Tester le backend	1 semaine	S7	S7	0	Oui
Tester le frontend	1 semaine	S7	S7	0	Oui
Vérifier les fonctionnalités	1 semaine	S7	S7	0	Oui
Corriger les bugs	1 semaine	S8	S8	0	Oui
Configurer le serveur Préparer la base de données	1 semaine	S9	S9	0	Oui
Mise en ligne	2 semaines	S10	S10	0	Oui
		S11	S11	0	Oui